บทที่ 4: การเข้ารหัสข้อมูลและการพัฒนาระบบให้เปลี่ยนแปลงได้

(สรุปเป็นภาษาไทยจากฉบับที่ 1 ใช้เวลาฟังประมาณ 15–17 นาที)

ตอนที่ 1: บทนำ

สวัสดีครับ ยินดีต้อนรับสู่บทสรุปหนังสือ Designing Data-Intensive Applications บทที่สี่ ว่าด้วยการเข้ารหัสข้อมูลและการพัฒนาระบบให้เปลี่ยนแปลงได้

บทที่หนึ่งเราพูดถึงเรื่องความพร้อมต่อการเปลี่ยนแปลง หรือ Evolvability บทนี้จะลงรายละเอียดเรื่องนี้ในมุมของข้อมูล

ระบบไม่ได้หยุดนิ่ง เมื่อเพิ่มความสามารถใหม่ ข้อมูลที่เก็บก็มักต้องเปลี่ยนตาม เช่น ต้องเพิ่มช่องข้อมูลใหม่ หรือเปลี่ยนรูปแบบข้อมูลเดิม

ปัญหาคือ ในระบบใหญ่ เราอัปเดตโค้ดทุกจุดพร้อมกันไม่ได้

ฝั่งเซิร์ฟเวอร์ มักทยอยอัปเดตทีละเครื่อง เพื่อไม่ให้ระบบหยุดให้บริการ วิธีนี้เรียกว่า rolling upgrade

ฝั่งผู้ใช้ยิ่งควบคุมไม่ได้ บางคนอาจไม่อัปเดตแอปเป็นเดือน ๆ

ช่วงเวลาหนึ่งจึงมีโค้ดเก่ากับโค้ดใหม่ทำงานอยู่ด้วยกัน และใช้ข้อมูลชุดเดียวกัน ระบบจึงต้องรองรับความเข้ากันได้สองทิศทาง

ทิศทางแรกเรียกว่า backward compatibility หรือความเข้ากันได้ย้อนหลัง หมายถึงโค้ดใหม่อ่านข้อมูลที่โค้ดเก่าเขียนไว้ได้ ข้อนี้ทำไม่ยาก เพราะตอนเขียนโค้ดใหม่ เรารู้อยู่แล้วว่าข้อมูลแบบเก่าหน้าตาเป็นอย่างไร

ทิศทางที่สองเรียกว่า forward compatibility หรือความเข้ากันได้ไปข้างหน้า หมายถึงโค้ดเก่าอ่านข้อมูลที่โค้ดใหม่เขียนได้ ข้อนี้ยากกว่า เพราะโค้ดเก่าต้องรู้จักมองข้ามสิ่งที่ตัวเองไม่รู้จัก

บทนี้จะพาไปดูรูปแบบการเข้ารหัสข้อมูลหลายแบบ ว่าแต่ละแบบรับมือเรื่องนี้ได้ดีแค่ไหน แล้วปิดท้ายด้วยวิธีที่ข้อมูลไหลไปมาระหว่างส่วนต่าง ๆ ของระบบครับ

ตอนที่ 2: การเข้ารหัสข้อมูลคืออะไร

โปรแกรมใช้ข้อมูลในสองรูปแบบ

รูปแบบแรกอยู่ในหน่วยความจำ เป็นออบเจกต์ ลิสต์ หรือโครงสร้างต่าง ๆ ซึ่งใช้ตัวชี้ เพื่อให้ซีพียูทำงานได้เร็ว

รูปแบบที่สองคือตอนส่งข้อมูลผ่านเครือข่ายหรือเขียนลงไฟล์ ข้อมูลต้องกลายเป็นชุดไบต์เรียงกัน เพราะตัวชี้จะไม่มีความหมายเลยเมื่อไปอยู่ในโปรแกรมอื่น

การแปลงจากรูปแบบในหน่วยความจำไปเป็นชุดไบต์ เรียกว่า encoding หรือ serialization ส่วนการแปลงกลับเรียกว่า decoding หรือ deserialization

ตอนที่ 3: รูปแบบเฉพาะของแต่ละภาษา

หลายภาษามีระบบเข้ารหัสข้อมูลติดมาให้ เช่น Java มี Serializable ส่วน Python มี pickle ใช้ง่ายมาก เขียนแค่บรรทัดเดียวก็บันทึกออบเจกต์ได้

แต่ผู้เขียนเตือนว่าไม่ควรใช้ นอกจากจะใช้ชั่วคราวจริง ๆ ด้วยเหตุผลสี่ข้อ

ข้อแรก ข้อมูลผูกติดกับภาษานั้น โปรแกรมที่เขียนด้วยภาษาอื่นอ่านได้ยากมาก

ข้อสอง มีปัญหาความปลอดภัย เพราะตอนถอดรหัส ระบบอาจสร้างออบเจกต์ชนิดใดก็ได้ตามที่ข้อมูลระบุ ถ้าผู้ไม่หวังดีส่งข้อมูลปลอมมา ก็อาจสั่งรันโค้ดอันตรายบนเครื่องของเราได้

ข้อสาม ส่วนใหญ่ไม่ได้ออกแบบมาเพื่อรองรับการเปลี่ยนเวอร์ชัน

ข้อสี่ ประสิทธิภาพมักไม่ดี เช่น ระบบของ Java ขึ้นชื่อว่าทั้งช้าและได้ข้อมูลขนาดใหญ่

ตอนที่ 4: JSON, XML และ CSV

ทางเลือกที่คนนิยมคือรูปแบบข้อความที่ใช้ได้ทุกภาษา เช่น JSON, XML และ CSV ซึ่งคนเปิดอ่านได้ด้วยตา แต่ก็มีปัญหาที่หลายคนไม่ทันสังเกต

ปัญหาแรกเรื่องตัวเลข XML กับ CSV แยกไม่ออกว่าข้อมูลเป็นตัวเลขหรือเป็นข้อความที่บังเอิญมีแต่ตัวเลข JSON แยกได้ แต่ไม่ได้แยกจำนวนเต็มกับทศนิยม และไม่ได้กำหนดความแม่นยำ

ตัวอย่างจริงคือ Twitter รหัสของแต่ละทวีตเป็นตัวเลขขนาดใหญ่มาก ใหญ่เกินกว่าที่ JavaScript จะเก็บได้แม่นยำ ตัวเลขจึงเพี้ยน Twitter แก้ด้วยการส่งรหัสไปสองแบบ ทั้งแบบตัวเลขและแบบข้อความ

ปัญหาที่สอง ทั้ง JSON และ XML เก็บข้อมูลไบนารี เช่น รูปภาพ ตรง ๆ ไม่ได้ ต้องแปลงเป็นข้อความด้วยวิธีที่เรียกว่า Base64 ซึ่งทำให้ข้อมูลใหญ่ขึ้นประมาณหนึ่งในสาม

ปัญหาที่สาม ระบบกำหนดโครงสร้างข้อมูล หรือสคีมา ของ JSON และ XML มีอยู่ แต่ซับซ้อนและไม่ได้ใช้กันแพร่หลาย ส่วน CSV ไม่มีสคีมาเลย และกฎก็คลุมเครือ เช่น ถ้าข้อมูลมีเครื่องหมายจุลภาคหรือขึ้นบรรทัดใหม่ จะจัดการอย่างไร แต่ละโปรแกรมทำไม่เหมือนกัน

ถึงอย่างนั้น ทั้งสามรูปแบบก็ยังเหมาะกับการแลกเปลี่ยนข้อมูลระหว่างองค์กร เพราะความยากที่สุดคือการทำให้หลายองค์กรตกลงใช้รูปแบบเดียวกัน ส่วนเรื่องรูปแบบจะสวยงามหรือมีประสิทธิภาพแค่ไหน กลายเป็นเรื่องรอง

ตอนที่ 5: การเข้ารหัสแบบไบนารี

ถ้าใช้ข้อมูลกันเองภายในองค์กร เราเลือกรูปแบบที่กระชับและเร็วกว่าได้ เรื่องนี้สำคัญมากเมื่อข้อมูลมีปริมาณระดับเทราไบต์

ผู้เขียนใช้ข้อมูลตัวอย่างชิ้นเดียวตลอดบท คือข้อมูลผู้ใช้ที่มีชื่อผู้ใช้ว่า Martin มีเลขที่ชอบคือ 1337 และมีงานอดิเรกสองอย่าง คือนั่งเหม่อฝันกลางวัน กับการแฮก

ถ้าเขียนเป็น JSON แบบไม่มีช่องว่าง จะได้ 81 ไบต์

มีรูปแบบไบนารีที่ดัดแปลงจาก JSON เช่น MessagePack ซึ่งเข้ารหัสข้อมูลเดียวกันได้ 66 ไบต์ ลดลงไม่มาก เพราะยังต้องเก็บชื่อช่องข้อมูลอย่าง userName ไว้ในทุกระเบียน

ตอนที่ 6: Thrift และ Protocol Buffers

Thrift มาจาก Facebook ส่วน Protocol Buffers หรือ protobuf มาจาก Google ทั้งสองเปิดเป็นโอเพนซอร์สช่วงปี 2007 ถึง 2008

ทั้งสองระบบต้องเขียนสคีมาไว้ก่อน โดยใช้ภาษากำหนดโครงสร้างที่เรียกว่า IDL ในสคีมาจะระบุชื่อช่องข้อมูล ชนิดข้อมูล และที่สำคัญที่สุดคือหมายเลขประจำช่อง เรียกว่า field tag เช่น ช่องชื่อผู้ใช้เป็นหมายเลขหนึ่ง ช่องเลขที่ชอบเป็นหมายเลขสอง

เวลาเข้ารหัส ระบบไม่ต้องเขียนชื่อช่องลงไป เขียนแค่หมายเลขซึ่งกินพื้นที่น้อยกว่ามาก

ผลลัพธ์คือ Thrift แบบ BinaryProtocol ได้ 59 ไบต์ Thrift แบบ CompactProtocol ได้ 34 ไบต์ และ Protocol Buffers ได้ 33 ไบต์ เล็กกว่า JSON เกินครึ่ง

แบบ CompactProtocol กับ protobuf เล็กได้ขนาดนี้ เพราะใช้เทคนิคเก็บตัวเลขแบบยืดหยุ่น ตัวเลขเล็กใช้ไบต์น้อย ตัวเลขใหญ่ค่อยใช้ไบต์มากขึ้น

หัวใจของการเปลี่ยนสคีมา คือหมายเลขประจำช่อง

เราเปลี่ยนชื่อช่องได้ตามสบาย เพราะในข้อมูลไม่มีชื่อช่องอยู่เลย แต่ห้ามเปลี่ยนหมายเลข เพราะจะทำให้ข้อมูลเก่าทั้งหมดอ่านผิด

ถ้าเพิ่มช่องใหม่ ต้องใช้หมายเลขใหม่ที่ไม่เคยใช้มาก่อน

โค้ดเก่าที่เจอหมายเลขที่ไม่รู้จัก ก็แค่ข้ามไป เพราะข้อมูลแต่ละช่องมีระบุชนิดไว้ ระบบจึงรู้ว่าต้องข้ามไปกี่ไบต์ แบบนี้ได้ความเข้ากันได้ไปข้างหน้า

ส่วนโค้ดใหม่ที่อ่านข้อมูลเก่า จะไม่เจอช่องใหม่ ดังนั้นช่องใหม่ต้องเป็นช่องที่ไม่บังคับ หรือมีค่าเริ่มต้น ถ้าตั้งเป็นช่องบังคับ โค้ดใหม่จะอ่านข้อมูลเก่าไม่ได้ แบบนี้ได้ความเข้ากันได้ย้อนหลัง

การลบช่องก็ใช้หลักเดียวกัน ลบได้เฉพาะช่องที่ไม่บังคับ และห้ามนำหมายเลขของช่องที่ลบไปใช้ซ้ำเด็ดขาด

การเปลี่ยนชนิดข้อมูลทำได้บางกรณี แต่ต้องระวัง เช่น ถ้าเปลี่ยนตัวเลขจาก 32 บิตเป็น 64 บิต โค้ดใหม่อ่านข้อมูลเก่าได้ แต่ถ้าโค้ดเก่าอ่านตัวเลขใหญ่ที่โค้ดใหม่เขียน ตัวเลขจะถูกตัดจนเพี้ยน

ตอนที่ 7: Avro

Avro เริ่มพัฒนาในปี 2009 เป็นโครงการย่อยของ Hadoop เพราะ Thrift ไม่ค่อยเหมาะกับงานของ Hadoop

Avro มีสคีมาเหมือนกัน แต่ไม่มีหมายเลขประจำช่อง ข้อมูลที่เข้ารหัสแล้วมีแค่ค่าต่าง ๆ เรียงต่อกัน ไม่มีทั้งชื่อช่อง ไม่มีทั้งชนิดข้อมูล ข้อมูลตัวอย่างของเราจึงเหลือแค่ 32 ไบต์ เล็กที่สุดในบรรดาทุกแบบ

คำถามคือ ถ้าในข้อมูลไม่มีอะไรบอกเลย จะอ่านถูกได้อย่างไร คำตอบคือ คนอ่านต้องใช้สคีมาเดียวกับที่คนเขียนใช้ตอนเขียนข้อมูล

สคีมาของผู้เขียน กับ สคีมาของผู้อ่าน

Avro แยกสคีมาเป็นสองชุด สคีมาที่ใช้ตอนเขียนข้อมูลเรียกว่า writer's schema ส่วนสคีมาที่โค้ดฝั่งอ่านคาดหวังเรียกว่า reader's schema

ทั้งสองไม่จำเป็นต้องเหมือนกัน แค่ต้องเข้ากันได้ ตอนอ่าน Avro จะเอาสองสคีมามาเทียบกัน แล้วจับคู่ช่องด้วยชื่อ ลำดับช่องจะต่างกันก็ไม่เป็นไร ถ้าข้อมูลมีช่องที่ฝั่งอ่านไม่รู้จัก ก็ทิ้งไป ถ้าฝั่งอ่านต้องการช่องที่ข้อมูลไม่มี ก็ใช้ค่าเริ่มต้นที่กำหนดไว้

ดังนั้น ใน Avro จะเพิ่มหรือลบช่องได้ก็ต่อเมื่อช่องนั้นมีค่าเริ่มต้น และถ้าอยากให้ช่องไหนเป็นค่าว่างได้ ต้องประกาศไว้ชัด ๆ ด้วยสิ่งที่เรียกว่า union type ซึ่งช่วยป้องกันบั๊กได้ด้วย

แล้วคนอ่านจะรู้สคีมาของคนเขียนได้อย่างไร

ขึ้นอยู่กับสถานการณ์

- ถ้าเป็นไฟล์ใหญ่ที่มีข้อมูลหลายล้านระเบียน ก็ใส่สคีมาไว้ตอนต้นไฟล์ครั้งเดียว
- ถ้าเป็นฐานข้อมูลที่เขียนทีละระเบียนในเวลาต่างกัน ก็ใส่หมายเลขเวอร์ชันไว้หน้าแต่ละระเบียน แล้วเก็บสคีมาทุกเวอร์ชันไว้ในที่เดียว
- ถ้าส่งข้อมูลผ่านเครือข่าย ทั้งสองฝั่งก็ตกลงเวอร์ชันสคีมากันตอนเริ่มเชื่อมต่อ

จุดเด่นที่สำคัญของ Avro

เมื่อไม่มีหมายเลขประจำช่อง เราจึงสร้างสคีมาอัตโนมัติจากข้อมูลได้ง่าย ตัวอย่างเช่น ถ้าอยากส่งออกข้อมูลจากฐานข้อมูลเชิงสัมพันธ์ ก็สร้างสคีมาจากโครงสร้างตารางได้เลย ถ้าวันหนึ่งตารางเปลี่ยน ก็สร้างสคีมาใหม่ได้ทันที

ถ้าเป็น Thrift หรือ protobuf ต้องมีคนคอยกำหนดหมายเลขช่องเองทุกครั้ง ซึ่งยุ่งยากและผิดพลาดได้ง่าย

ข้อดีของการมีสคีมา

ผู้เขียนสรุปว่า รูปแบบไบนารีที่มีสคีมามีข้อดีหลายอย่าง

- ข้อมูลเล็กกว่ามาก เพราะไม่ต้องเก็บชื่อช่อง
- สคีมาเป็นเอกสารอธิบายข้อมูลไปในตัว และไม่มีวันล้าสมัย เพราะถ้าไม่ตรงกับข้อมูลจริง ระบบก็ใช้งานไม่ได้
- เก็บสคีมาทุกเวอร์ชันไว้ แล้วตรวจได้ก่อนปล่อยระบบ ว่าเวอร์ชันใหม่เข้ากับเวอร์ชันเก่าหรือไม่
- ภาษาที่ตรวจชนิดข้อมูลตอนคอมไพล์ สร้างโค้ดจากสคีมาได้อัตโนมัติ

แนวคิดนี้ไม่ได้ใหม่ มาตรฐานชื่อ ASN.1 ที่เกิดขึ้นตั้งแต่ปี 1984 ก็ใช้แนวคิดเดียวกัน และยังใช้อยู่ในใบรับรองความปลอดภัยของเว็บไซต์จนถึงวันนี้

ตอนที่ 8: ข้อมูลไหลผ่านฐานข้อมูล

ทีนี้มาดูว่าข้อมูลเดินทางจากโปรแกรมหนึ่งไปอีกโปรแกรมหนึ่งได้อย่างไร มีสามช่องทางหลัก คือผ่านฐานข้อมูล ผ่านการเรียกบริการ และผ่านการส่งข้อความ

เริ่มจากฐานข้อมูล การเขียนลงฐานข้อมูลก็เหมือนการส่งข้อความไปหาตัวเองในอนาคต ความเข้ากันได้ย้อนหลังจึงจำเป็นไม่อย่างนั้นตัวเราในอนาคตจะอ่านข้อมูลที่เขียนไว้ไม่ออก

มีกับดักที่ควรรู้ สมมติโค้ดใหม่เพิ่มช่องข้อมูลใหม่ แล้วเขียนข้อมูลลงไป ต่อมาโค้ดเก่าที่ยังทำงานอยู่ อ่านข้อมูลนั้นมาแก้ไข แล้วเขียนกลับลงไป ถ้าโค้ดเก่าไม่ระวัง ช่องใหม่ที่มันไม่รู้จักจะหายไปโดยไม่มีใครรู้ ดังนั้นโค้ดต้องเก็บช่องที่ไม่รู้จักไว้ให้ครบตอนเขียนกลับ

อีกเรื่องหนึ่งคือ ข้อมูลมีอายุยืนกว่าโค้ด ภายในไม่กี่นาที เราอัปเดตโค้ดทั้งระบบได้ แต่ข้อมูลที่เขียนไว้เมื่อห้าปีก่อนก็ยังอยู่ในรูปแบบเดิม

การแปลงข้อมูลเก่าทั้งหมดให้เป็นรูปแบบใหม่ทำได้ แต่สำหรับข้อมูลขนาดใหญ่ ใช้ทั้งเวลาและทรัพยากรมาก ฐานข้อมูลส่วนใหญ่จึงหลีกเลี่ยง เช่น เมื่อเพิ่มคอลัมน์ใหม่ ฐานข้อมูลไม่ได้แก้ทุกแถว แต่จะเติมค่าว่างให้เองตอนอ่านแถวเก่า

ส่วนเวลาสำรองข้อมูลเก็บไว้ เราเข้ารหัสข้อมูลทั้งหมดด้วยสคีมาล่าสุดได้เลย เพราะเป็นการคัดลอกทั้งหมดในครั้งเดียว และรูปแบบแบบคอลัมน์ที่พูดถึงในบทที่สาม เช่น Parquet ก็เหมาะกับงานนี้มาก

ตอนที่ 9: ข้อมูลไหลผ่านการเรียกบริการ

ช่องทางที่สองคือการสื่อสารผ่านเครือข่าย ฝั่งหนึ่งเรียกว่าไคลเอนต์ ทำหน้าที่ส่งคำขอ อีกฝั่งเรียกว่าเซิร์ฟเวอร์ ทำหน้าที่ให้บริการ

ตัวอย่างที่คุ้นเคยที่สุดคือเว็บเบราว์เซอร์คุยกับเว็บเซิร์ฟเวอร์ แต่ในระบบใหญ่ เซิร์ฟเวอร์เองก็เรียกใช้บริการอื่นต่ออีกทอดการแบ่งระบบใหญ่เป็นบริการเล็ก ๆ หลายตัวแบบนี้ เรียกว่า microservices

ข้อดีคือแต่ละทีมดูแลบริการของตัวเองและปล่อยเวอร์ชันใหม่เองได้ โดยไม่ต้องรอทีมอื่น ดังนั้นโค้ดเก่ากับโค้ดใหม่จึงต้องคุยกันรู้เรื่องเสมอ

REST และ SOAP

REST ไม่ใช่โปรโตคอล แต่เป็นแนวคิดการออกแบบที่ใช้ความสามารถของเว็บตามปกติ เช่น URL และคำสั่งของ HTTP ข้อมูลมักเป็น JSON ใช้ง่ายและเป็นที่นิยมมาก

SOAP เป็นโปรโตคอลที่ใช้ XML ซึ่งมีมาตรฐานประกอบเยอะและซับซ้อนมาก ต้องพึ่งเครื่องมือช่วย ปัจจุบันจึงได้รับความนิยมน้อยลง ยกเว้นในองค์กรขนาดใหญ่บางแห่ง

ปัญหาของ RPC

RPC ย่อมาจาก Remote Procedure Call เป็นแนวคิดที่มีมาตั้งแต่ปี 1970 คือทำให้การเรียกบริการผ่านเครือข่าย ดูเหมือนการเรียกฟังก์ชันธรรมดาในโปรแกรม

ฟังดูสะดวก แต่ผู้เขียนบอกว่าแนวคิดนี้มีข้อบกพร่องพื้นฐาน เพราะการเรียกผ่านเครือข่ายต่างจากการเรียกฟังก์ชันในเครื่องมาก

- ฟังก์ชันในเครื่องคาดเดาได้ แต่คำขอผ่านเครือข่ายอาจหาย หรือเครื่องปลายทางอาจช้าหรือล่มก็ได้
- คำขอผ่านเครือข่ายอาจหมดเวลาโดยไม่ได้คำตอบ เราจะไม่รู้เลยว่าอีกฝั่งทำงานไปแล้วหรือยัง
- ถ้าส่งคำขอซ้ำ งานอาจถูกทำสองครั้ง เช่น ตัดเงินลูกค้าสองรอบ ต้องออกแบบให้ทำซ้ำได้โดยผลลัพธ์ไม่เปลี่ยน ซึ่งเรียกว่า idempotence
- เวลาที่ใช้ไม่แน่นอน บางครั้งไม่ถึงหนึ่งมิลลิวินาที บางครั้งหลายวินาที
- ส่งตัวชี้ไปยังข้อมูลในหน่วยความจำไม่ได้ ต้องเข้ารหัสข้อมูลทั้งหมดส่งไป
- ถ้าสองฝั่งใช้ภาษาต่างกัน ชนิดข้อมูลอาจแปลงกันได้ไม่ตรง

เฟรมเวิร์ก RPC รุ่นใหม่ เช่น gRPC ที่ใช้ Protocol Buffers ยอมรับความจริงข้อนี้ จึงไม่พยายามซ่อนว่าเป็นการเรียกผ่านเครือข่าย และมีเครื่องมืออย่าง future หรือ promise สำหรับรอผลลัพธ์ที่อาจล้มเหลวได้

ข้อดีของ RPC แบบไบนารีคือเร็วกว่า เหมาะกับการคุยกันระหว่างบริการภายในองค์กร ส่วน REST เหมาะกับบริการที่เปิดให้คนภายนอกใช้ เพราะทดสอบง่าย แค่เปิดเบราว์เซอร์ก็ลองได้

การเปลี่ยนเวอร์ชันของบริการ

ปกติเราจะอัปเดตเซิร์ฟเวอร์ก่อน แล้วไคลเอนต์ค่อยตามมาทีหลัง ดังนั้นคำขอต้องเข้ากันได้ย้อนหลัง คือเซิร์ฟเวอร์ใหม่ต้องรับคำขอแบบเก่าได้ ส่วนคำตอบต้องเข้ากันได้ไปข้างหน้า คือไคลเอนต์เก่าต้องอ่านคำตอบแบบใหม่ได้

บริการที่เปิดให้คนภายนอกใช้ ไม่มีทางบังคับให้ลูกค้าอัปเดตได้ จึงมักต้องเปิดหลายเวอร์ชันไว้พร้อมกันนาน ๆ โดยใส่หมายเลขเวอร์ชันไว้ใน URL หรือใน HTTP header

ตอนที่ 10: ข้อมูลไหลผ่านการส่งข้อความ

ช่องทางที่สามอยู่ตรงกลางระหว่างสองแบบแรก คือส่งข้อความผ่านตัวกลางที่เรียกว่า message broker เช่น RabbitMQ, ActiveMQ และ Apache Kafka

ผู้ส่งไม่ได้ส่งข้อความหาผู้รับโดยตรง แต่ฝากไว้กับตัวกลาง แล้วตัวกลางส่งต่อให้ผู้รับ เหมือนการฝากจดหมายไว้ที่ไปรษณีย์

วิธีนี้มีข้อดีหลายอย่าง

- ถ้าผู้รับยุ่งหรือล่มอยู่ ตัวกลางก็ช่วยพักข้อความไว้ก่อน
- ถ้าผู้รับล่มกลางทาง ตัวกลางส่งข้อความซ้ำให้ใหม่ได้ ข้อความจึงไม่หาย
- ผู้ส่งไม่ต้องรู้ที่อยู่ของผู้รับ
- ข้อความเดียวส่งให้ผู้รับหลายรายได้
- ผู้ส่งกับผู้รับแยกจากกันชัดเจน ต่างฝ่ายต่างพัฒนาได้อิสระ

ข้อแตกต่างสำคัญจาก RPC คือการส่งข้อความเป็นแบบทางเดียว ผู้ส่งไม่รอคำตอบ ถ้าต้องการคำตอบ ผู้รับก็ต้องส่งข้อความกลับมาอีกช่องทางหนึ่ง

ตัวกลางส่วนใหญ่ไม่สนใจว่าข้อมูลข้างในเป็นรูปแบบไหน ไบต์อะไรก็ส่งได้ เราจึงเลือกใช้รูปแบบที่รองรับการเปลี่ยนเวอร์ชันได้ตามที่ต้องการ แต่ก็ต้องระวังปัญหาเดียวกับฐานข้อมูล คือถ้าโค้ดเก่ารับข้อความแล้วส่งต่อ ต้องไม่ทำช่องข้อมูลใหม่หาย

Actor Model

อีกแนวคิดหนึ่งคือ actor model แทนที่จะให้หลายเธรดแย่งกันใช้ข้อมูล โปรแกรมจะแบ่งเป็นตัวละครเล็ก ๆ ที่เรียกว่า actor แต่ละตัวมีข้อมูลของตัวเอง และคุยกับตัวอื่นด้วยการส่งข้อความเท่านั้น

เฟรมเวิร์กอย่าง Akka, Orleans และ Erlang OTP ขยายแนวคิดนี้ไปทำงานข้ามหลายเครื่องได้ แต่ก็ยังต้องคิดเรื่องความเข้ากันได้ของข้อความเวลาทยอยอัปเดตระบบเหมือนกัน

ตอนที่ 11: สรุปท้ายบท

มาทบทวนกันอีกครั้งครับ

หนึ่ง ระบบจริงต้องทยอยอัปเดต จึงมีโค้ดเก่ากับโค้ดใหม่ทำงานพร้อมกันเสมอ ข้อมูลจึงต้องเข้ากันได้ทั้งย้อนหลังและไปข้างหน้า

สอง รูปแบบเฉพาะของแต่ละภาษา เช่น pickle ของ Python ใช้งานง่าย แต่ผูกติดกับภาษา ไม่ปลอดภัย และไม่รองรับการเปลี่ยนเวอร์ชัน

สาม JSON, XML และ CSV ใช้ได้กว้างขวาง แต่คลุมเครือเรื่องชนิดข้อมูล โดยเฉพาะตัวเลข

สี่ รูปแบบไบนารีที่มีสคีมา อย่าง Thrift, Protocol Buffers และ Avro ทั้งกระชับ มีประสิทธิภาพ และรองรับการเปลี่ยนเวอร์ชันได้ดี ถ้าทำตามกฎ Thrift กับ protobuf ใช้หมายเลขประจำช่อง ส่วน Avro ใช้การเทียบสคีมาของผู้เขียนกับผู้อ่าน

ห้า ข้อมูลไหลได้สามช่องทาง ผ่านฐานข้อมูล ผ่านการเรียกบริการแบบ REST หรือ RPC และผ่านการส่งข้อความด้วยตัวกลาง ทุกช่องทางต้องคิดเรื่องความเข้ากันได้ทั้งหมด

ด้วยเครื่องมือที่ถูกต้องและความรอบคอบเล็กน้อย เราอัปเดตระบบได้บ่อย ๆ โดยไม่ต้องหยุดให้บริการ และนี่คือหัวใจของระบบที่พร้อมต่อการเปลี่ยนแปลง

บทนี้เป็นบทสุดท้ายของภาคที่หนึ่ง ซึ่งพูดถึงพื้นฐานของระบบข้อมูลบนเครื่องเดียว บทต่อไปคือบทที่ห้า เป็นจุดเริ่มต้นของภาคที่สอง ว่าด้วยข้อมูลแบบกระจาย โดยเริ่มจากเรื่องการทำสำเนาข้อมูล หรือ Replication ขอบคุณที่ฟังครับ

ทิปสำหรับ Edge: บทนี้มีตัวเลขขนาดข้อมูลหลายจุด (81, 66, 59, 34, 33 และ 32 ไบต์) ถ้าฟังอย่างเดียวแล้วจำไม่ได้ ให้จำแค่ว่า JSON ใหญ่สุด ส่วน Avro เล็กสุดก็พอครับ

บทที่ 4 ปิดภาคที่ 1 แล้ว จะให้ทำบทที่ 5 เรื่อง Replication ต่อเลยไหมครับ หรือจะรวมบทที่ 1–4 เป็นไฟล์ PDF ไว้เปิดใน Edge ก็ได้ 😊